

roots. You can start with the one root of $p_1(x)$ and then, in turn, bracket the roots of each higher j , pinning them down at each stage more precisely by Newton's rule or some other root-finding scheme (see Chapter 9).

Why would you ever want to find all the roots of an orthogonal polynomial $p_j(x)$? Because the abscissas of the N -point Gaussian quadrature formulas (4.6.1) and (4.6.4) with weighting function $W(x)$ in the interval (a, b) are precisely the roots of the orthogonal polynomial $p_N(x)$ for the same interval and weighting function. This is the fundamental theorem of Gaussian quadratures, and it lets you find the abscissas for any particular case.

Once you know the abscissas $x_0, \dots, x_{N-1}$, you need to find the weights w_j , $j = 0, \dots, N-1$. One way to do this (not the most efficient) is to solve the set of linear equations

$$\begin{bmatrix} p_0(x_0) & \dots & p_0(x_{N-1}) \\ p_1(x_0) & \dots & p_1(x_{N-1}) \\ \vdots & & \vdots \\ p_{N-1}(x_0) & \dots & p_{N-1}(x_{N-1}) \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{N-1} \end{bmatrix} = \begin{bmatrix} \int_a^b W(x) p_0(x) dx \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (4.6.8)$$

Equation (4.6.8) simply solves for those weights such that the quadrature (4.6.1) gives the correct answer for the integral of the first N orthogonal polynomials. Note that the zeros on the right-hand side of (4.6.8) appear because $p_1(x), \dots, p_{N-1}(x)$ are all orthogonal to $p_0(x)$, which is a constant. It can be shown that, with those weights, the integral of the *next* $N-1$ polynomials is also exact, so that the quadrature is exact for all polynomials of degree $2N-1$ or less. Another way to evaluate the weights (though one whose proof is beyond our scope) is by the formula

$$w_j = \frac{\langle p_{N-1} | p_{N-1} \rangle}{p_{N-1}(x_j) p'_N(x_j)} \quad (4.6.9)$$

where $p'_N(x_j)$ is the derivative of the orthogonal polynomial at its zero x_j .

The computation of Gaussian quadrature rules thus involves two distinct phases: (i) the generation of the orthogonal polynomials $p_0, \dots, p_N$, i.e., the computation of the coefficients a_j, b_j in (4.6.6), and (ii) the determination of the zeros of $p_N(x)$, and the computation of the associated weights. For the case of the “classical” orthogonal polynomials, the coefficients a_j and b_j are explicitly known (equations 4.6.10 – 4.6.14 below) and phase (i) can be omitted. However, if you are confronted with a “nonclassical” weight function $W(x)$, and you don't know the coefficients a_j and b_j , the construction of the associated set of orthogonal polynomials is not trivial. We discuss it at the end of this section.

4.6.1 Computation of the Abscissas and Weights

This task can range from easy to difficult, depending on how much you already know about your weight function and its associated polynomials. In the case of classical, well-studied, orthogonal polynomials, practically everything is known, including good approximations for their zeros. These can be used as starting guesses, enabling Newton's method (to be discussed in §9.4) to converge very rapidly. Newton's method requires the derivative $p'_N(x)$, which is evaluated by standard relations in terms of p_N and p_{N-1} . The weights are then conveniently evaluated by equation

(4.6.9). For the following named cases, this direct root finding is faster, by a factor of 3 to 5, than any other method.

Here are the weight functions, intervals, and recurrence relations that generate the most commonly used orthogonal polynomials and their corresponding Gaussian quadrature formulas.

Gauss-Legendre:

$$\begin{aligned} W(x) &= 1 & -1 < x < 1 \\ (j+1)P_{j+1} &= (2j+1)xP_j - jP_{j-1} \end{aligned} \quad (4.6.10)$$

Gauss-Chebyshev:

$$\begin{aligned} W(x) &= (1-x^2)^{-1/2} & -1 < x < 1 \\ T_{j+1} &= 2xT_j - T_{j-1} \end{aligned} \quad (4.6.11)$$

Gauss-Laguerre:

$$\begin{aligned} W(x) &= x^\alpha e^{-x} & 0 < x < \infty \\ (j+1)L_{j+1}^\alpha &= (-x+2j+\alpha+1)L_j^\alpha - (j+\alpha)L_{j-1}^\alpha \end{aligned} \quad (4.6.12)$$

Gauss-Hermite:

$$\begin{aligned} W(x) &= e^{-x^2} & -\infty < x < \infty \\ H_{j+1} &= 2xH_j - 2jH_{j-1} \end{aligned} \quad (4.6.13)$$

Gauss-Jacobi:

$$\begin{aligned} W(x) &= (1-x)^\alpha (1+x)^\beta & -1 < x < 1 \\ c_j P_{j+1}^{(\alpha,\beta)} &= (d_j + e_j x) P_j^{(\alpha,\beta)} - f_j P_{j-1}^{(\alpha,\beta)} \end{aligned} \quad (4.6.14)$$

where the coefficients c_j , d_j , e_j , and f_j are given by

$$\begin{aligned} c_j &= 2(j+1)(j+\alpha+\beta+1)(2j+\alpha+\beta) \\ d_j &= (2j+\alpha+\beta+1)(\alpha^2-\beta^2) \\ e_j &= (2j+\alpha+\beta)(2j+\alpha+\beta+1)(2j+\alpha+\beta+2) \\ f_j &= 2(j+\alpha)(j+\beta)(2j+\alpha+\beta+2) \end{aligned} \quad (4.6.15)$$

We now give individual routines that calculate the abscissas and weights for these cases. First comes the most common set of abscissas and weights, those of Gauss-Legendre. The routine, due to G.B. Rybicki, uses equation (4.6.9) in the special form for the Gauss-Legendre case,

$$w_j = \frac{2}{(1-x_j^2)[P_N'(x_j)]^2} \quad (4.6.16)$$

The routine also scales the range of integration from (x_1, x_2) to $(-1, 1)$, and provides abscissas x_j and weights w_j for the Gaussian formula

$$\int_{x_1}^{x_2} f(x)dx = \sum_{j=0}^{N-1} w_j f(x_j) \quad (4.6.17)$$

gauss_wgts.h

```

void gauleg(const Doub x1, const Doub x2, VecDoub_0 &x, VecDoub_0 &w)
Given the lower and upper limits of integration x1 and x2, this routine returns arrays x[0..n-1]
and w[0..n-1] of length n, containing the abscissas and weights of the Gauss-Legendre n-point
quadrature formula.
{
    const Doub EPS=1.0e-14;                               EPS is the relative precision.
    Doub z1,z,xm,xl,pp,p3,p2,p1;
    Int n=x.size();
    Int m=(n+1)/2;                                         The roots are symmetric in the interval, so
                                                            we only have to find half of them.
    xm=0.5*(x2+x1);
    xl=0.5*(x2-x1);
    for (Int i=0;i<m;i++) {                                Loop over the desired roots.
        z=cos(3.141592654*(i+0.75)/(n+0.5));
        Starting with this approximation to the ith root, we enter the main loop of refinement
        by Newton's method.
        do {
            p1=1.0;
            p2=0.0;
            for (Int j=0;j<n;j++) {                          Loop up the recurrence relation to get the
                                                            Legendre polynomial evaluated at z.
                p3=p2;
                p2=p1;
                p1=((2.0*j+1.0)*z*p2-j*p3)/(j+1);
            }
            p1 is now the desired Legendre polynomial. We next compute pp, its derivative,
            by a standard relation involving also p2, the polynomial of one lower order.
            pp=n*(z*p1-p2)/(z*z-1.0);
            z1=z;
            z=z1-p1/pp;                                       Newton's method.
        } while (abs(z-z1) > EPS);
        x[i]=xm-xl*z;
        x[n-1-i]=xm+xl*z;
        w[i]=2.0*xl/((1.0-z*z)*pp*pp);                       Compute the weight
        w[n-1-i]=w[i];                                       and its symmetric counterpart.
    }
}

```

Next we give three routines that use initial approximations for the roots given by Stroud and Secrest [2]. The first is for Gauss-Laguerre abscissas and weights, to be used with the integration formula

$$\int_0^{\infty} x^{\alpha} e^{-x} f(x) dx = \sum_{j=0}^{N-1} w_j f(x_j) \quad (4.6.18)$$

gauss_wgts.h

```

void gaulag(VecDoub_0 &x, VecDoub_0 &w, const Doub alf)
Given alf, the parameter  $\alpha$  of the Laguerre polynomials, this routine returns arrays x[0..n-1]
and w[0..n-1] containing the abscissas and weights of the n-point Gauss-Laguerre quadrature
formula. The smallest abscissa is returned in x[0], the largest in x[n-1].
{
    const Int MAXIT=10;
    const Doub EPS=1.0e-14;                               EPS is the relative precision.
    Int i,its,j;
    Doub ai,p1,p2,p3,pp,z,z1;
    Int n=x.size();
    for (i=0;i<n;i++) {                                    Loop over the desired roots.
        if (i == 0) {                                       Initial guess for the smallest root.
            z=(1.0+alf)*(3.0+0.92*alf)/(1.0+2.4*n+1.8*alf);
        } else if (i == 1) {                                Initial guess for the second root.
            z += (15.0+6.25*alf)/(1.0+0.9*alf+2.5*n);
        } else {                                           Initial guess for the other roots.
            ai=i-1;
        }
    }
}

```